CLUSTER ANALYSIS USING PYTHON

A Beginner-Friendly Practical

Four different clustering methods on one dataset

Agglomerative • Divisive • K-Means • DBSCAN

Name: _____ Roll No.: _____ Date: _____

Start Here

AIM
To perform four different types of cluster analysis in Python on **one** customer dataset, and to compare the results.

What you need

- Python with Jupyter Notebook, Google Colab or any Python editor.
- Install the libraries once (in a terminal): `pip install pandas numpy matplotlib scikit-learn scipy`
- The data file `customer_segmentation.csv` saved in **the same folder** as your notebook (in Colab: upload it using the folder icon on the left).

How to work through this manual

1. Read the short **idea in plain words** for each part.
2. Copy the code block into a notebook cell and **run it**. Run the blocks **in order** – each one uses results from the one before.
3. Compare your screen with **What you should see**.
4. Do the **Your turn** task and write your answer in your notebook.

NOTE
Every code block has a comment (# text) on almost every line. Read them – they explain what each line does. Lines marked `# <-- YOUR DECISION` are choices you must make yourself.

1. What Is Clustering?

Imagine a shop owner with 450 customers. She cannot send a different offer to each one, but she also does not want to send the same offer to everyone. **Clustering** sorts the customers into **groups of similar customers** automatically – without anyone telling the computer in advance what the groups are. Because there is no "right answer" column to learn from, this is called **unsupervised learning**.

How does the computer know who is "similar"?

It measures **distance**. Think of each customer as a point on a map; customers who are close together are similar. With six variables the map has six directions instead of two, but the idea is the same. The most common measure is **Euclidean distance** – the ordinary straight-line distance.

The four methods in this practical

Method	Idea in one sentence	Everyday picture
Agglomerative	Start with every customer alone, then keep joining the two closest groups.	Friends joining into bigger and bigger friend circles.
Divisive	Start with everyone in one group, then keep splitting it.	A big class being divided into smaller and smaller teams.
K-Means	You choose the number of groups (K); the computer finds K centre points and puts each customer with the nearest one.	Choosing K meeting points and everyone walks to the nearest.
DBSCAN	Groups are crowded areas; customers in empty areas are marked as "noise".	Crowds in a park, with a few people standing alone.

2. The Dataset

The file has **453 rows** (customers) and **9 columns**. After removing 3 repeated rows we have **450 customers**. The data is a bit messy on purpose – as real data is.

Column	Meaning	Used for clustering?
Customer_ID	Customer's code (e.g. C1154)	**No** – it is just a name tag
Age	Age in years	Yes
Annual_Income	Income per year	Yes
Spending_Score	Score from 1 to 100 for how much the customer spends	Yes
Purchase_Frequency	Purchases per month	Yes
Avg_Purchase_Value	Average amount spent per purchase	Yes
Online_Purchase_Ratio	Share of purchases made online (0 to 1)	Yes
Category	Favourite product type (words)	No – used later to describe groups
Membership	Basic / Silver / Gold (words)	No – used later to describe groups

Why only the numerical columns? Clustering uses distances, and distances need numbers. The word columns cannot be subtracted from each other, so we leave them out of the calculation. Why Customer_ID is excluded: a code like C1154 means nothing about behaviour, so including it would only add random noise.

Part A. Get the Data Ready

Step 1: Load the tools and the data

Idea in plain words: Python needs to be told which tools to use (*import*), and then we read the file into a table called `df`.

```
# ===== PART A, STEP 1: Load the tools and the data =====
import pandas as pd                      # tables of data
import numpy as np                       # numbers and arrays
import matplotlib.pyplot as plt          # drawing graphs
from sklearn.preprocessing import StandardScaler          # puts variables on the same scale
from sklearn.cluster import AgglomerativeClustering, KMeans, DBSCAN   # the clustering tools
from sklearn.metrics import silhouette_score               # a score for cluster quality
from sklearn.decomposition import PCA                      # squeezes data into 2D for plotting
from scipy.cluster.hierarchy import linkage, dendrogram    # for the dendrogram (tree)

pd.set_option("display.width", 200)         # show wide tables without "..."
pd.set_option("display.max_columns", None)

df = pd.read_csv("customer_segmentation.csv")   # read the file into a table called df

print(df.head())      # look at the first 5 customers
print(df.shape)       # (number of rows, number of columns)
print(df.isnull().sum())   # how many missing values in each column?
```

What you should see: the first five customers; the shape `(453, 9)`; and a list showing missing values in Annual_Income (22), Spending_Score (18), Avg_Purchase_Value (18), Online_Purchase_Ratio (14) and Category (9).

STUDENT TASK

- Write down: number of customers, number of columns, and which columns have missing values.

Step 2: Clean the data

Idea in plain words: Clustering cannot work with empty cells, and a customer listed twice would count double. We remove repeats and fill each empty number with the **median** (the middle value of the

column). We prefer the median to the average because a few very rich customers pull the average upwards.

```
# ===== PART A, STEP 2: Clean the data =====
# 1. Remove customers who appear twice
df = df.drop_duplicates().reset_index(drop=True)

# 2. The columns we will use for clustering (all numbers)
features = ["Age", "Annual_Income", "Spending_Score",
            "Purchase_Frequency", "Avg_Purchase_Value", "Online_Purchase_Ratio"]

# 3. Fill each missing number with the MEDIAN (the "middle" value) of its column
for col in features:
    df[col] = df[col].fillna(df[col].median())

print("Customers:", len(df))
print("Missing values left:", df[features].isnull().sum().sum())
```

```
Customers: 450
Missing values left: 0
```

STUDENT TASK

- Explain in one sentence why the median is a safer choice than the mean for Annual_Income.

Step 3: Put all variables on the same scale

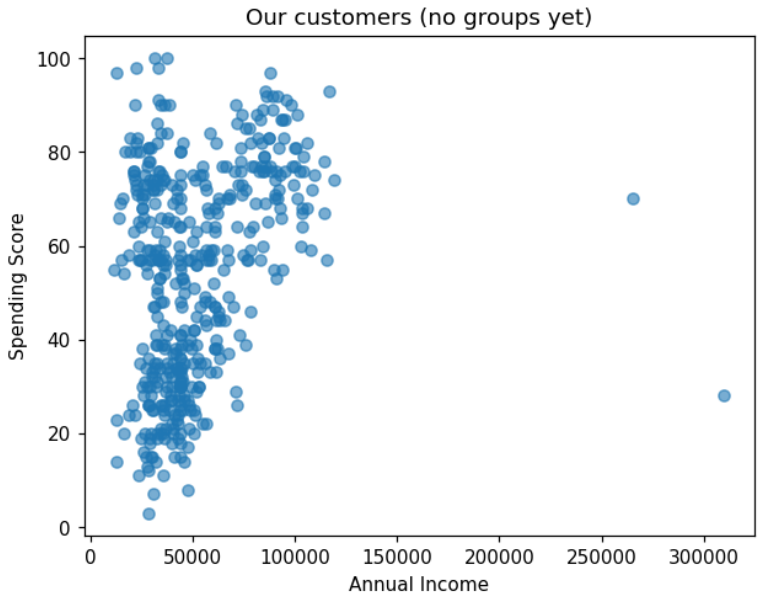
Idea in plain words: Income is measured in thousands (e.g. 52,000) but the online ratio is between 0 and 1. If we measured distance directly, **income would completely overpower** every other variable – like judging a person's overall fitness only by their weight in grams. **Standardization** rescales every variable so that its average is 0 and its spread is 1, giving each variable a fair say.

```
# ===== PART A, STEP 3: Put all variables on the same scale =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])   # every column now has mean 0, spread 1

# Check: means should be about 0 and standard deviations about 1
print(pd.DataFrame(X_scaled, columns=features).describe().loc[["mean", "std"]].round(2))

# A first picture of the data (income vs spending)
plt.scatter(df["Annual_Income"], df["Spending_Score"], alpha=0.6)
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Our customers (no groups yet)")
plt.show()
```

What you should see: a table where every mean is about 0 and every standard deviation is 1, and a scatter plot of income against spending. Do you see clear separate clouds, or one blurry cloud? (Real data is usually blurry!)



Sample output: customers plotted by income and spending, before any clustering.

STUDENT TASK

- Write down how many groups you **expect** to find. You will compare with your final answer.

Part B. Method 1: Agglomerative Clustering

Idea in plain words (bottom-up):

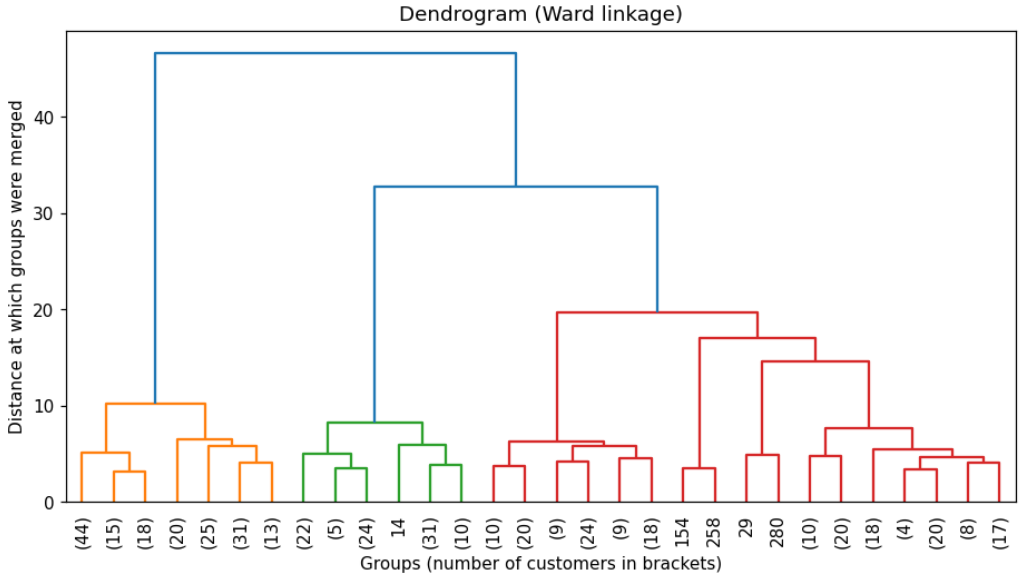
1. Every customer starts as a group of one (450 groups).
2. Find the two closest groups and join them.
3. Repeat until only one big group remains.
4. The history of all the joins is drawn as a tree called a **dendrogram**. You then "cut" the tree to get the number of groups you want.

Linkage is the rule for measuring the distance between two *groups*. We use **Ward**, which joins the groups that make the new group as compact as possible. (Other rules: single = nearest members, complete = farthest members, average = average of all pairs.)

```
# ===== PART B: Agglomerative (bottom-up) hierarchical clustering =====
# Step 1: build the tree and draw it (Ward linkage makes compact groups)
tree = linkage(X_scaled, method="ward")
plt.figure(figsize=(10, 5))
dendrogram(tree, truncate_mode="lastp", p=30, leaf_rotation=90)   # show the last 30 merges
plt.title("Dendrogram (Ward linkage)")
plt.xlabel("Groups (number of customers in brackets)")
plt.ylabel("Distance at which groups were merged")
plt.show()

# Step 2: decide how many groups YOU see, then cut the tree there
K = 4        # <-- YOUR DECISION: change after reading the dendrogram
agg = AgglomerativeClustering(n_clusters=K, linkage="ward")
agg_labels = agg.fit_predict(X_scaled)           # a group number (0,1,2,..) for each customer

print("Customers per group:")
print(pd.Series(agg_labels).value_counts().sort_index())
```



Sample output: dendrogram with Ward linkage.

How to read a dendrogram

- The **bottom** shows customers (in bracketed lumps); the **height** shows how far apart two groups were when they were joined.
- **Tall vertical lines** are joins between very different groups. Imagine a horizontal line cutting through the tallest vertical lines: **the number of lines it crosses is the number of groups.**

What you should see: a printed count of customers in each group (for K = 4 the sample run gave 101, 166, 93 and 90).

STUDENT TASK

- Look at your dendrogram. How many groups would you choose? Change K to your number and run the block again. Explain your choice in 2–3 lines.

Part C. Method 2: Divisive Clustering

Idea in plain words (top-down): the opposite of Part B. Start with **everyone in one group** and keep splitting.

1. Put all customers in one group.
2. Find the group that is the **most spread out**.
3. Split it into two using K-means with 2 groups.
4. Repeat until you have K groups.

NOTE

Python's scikit-learn does not have a ready-made divisive clustering command, so we write a short function ourselves. This is a **teaching version** (it splits using K-means). Other divisive methods exist.

```
# ===== PART C: Divisive (top-down) clustering =====
# Start with ONE big group, then keep splitting the most spread-out group in two.
def divisive(X, n_groups):
    labels = np.zeros(len(X), dtype=int)           # everyone starts in group 0
    while len(np.unique(labels)) < n_groups:
        # 1. find the group that is the most spread out
        worst_group, worst_spread = None, -1
        for g in np.unique(labels):
            points = X[labels == g]
            spread = ((points - points.mean(axis=0)) ** 2).sum()   # total squared distance from centre
            if spread > worst_spread and len(points) > 1:
                worst_group, worst_spread = g, spread
        # 2. split that group into two using K-means with 2 groups
        members = np.where(labels == worst_group)[0]
        halves = KMeans(n_clusters=2, n_init=10, random_state=42).fit_predict(X[members])
        # 3. one half keeps the old number, the other half gets a new number
        labels[members[halves == 1]] = labels.max() + 1
        print(f"Split group {worst_group}: now {len(np.unique(labels))} groups")
    return labels

div_labels = divisive(X_scaled, K)
print(pd.Series(div_labels).value_counts().sort_index())

Split group 0: now 2 groups
Split group 0: now 3 groups
Split group 0: now 4 groups
0     77
1    172
2    102
3     99
```

STUDENT TASK

- Run it with 2, 3 and 5 groups (change K or call `divisive(X_scaled, 3)`). Compare group sizes with Part B. Are they similar?

Part D. Method 3: K-Means

Idea in plain words:

1 **You** choose K, the number of groups.
2 The computer places K centre points (**centroids**) on the map.
3 Every customer joins the **nearest** centroid.
4 Each centroid moves to the **average position** of its customers.
5 Repeat steps 3–4 until nothing changes.

But how do I choose K?

We use two helpers:

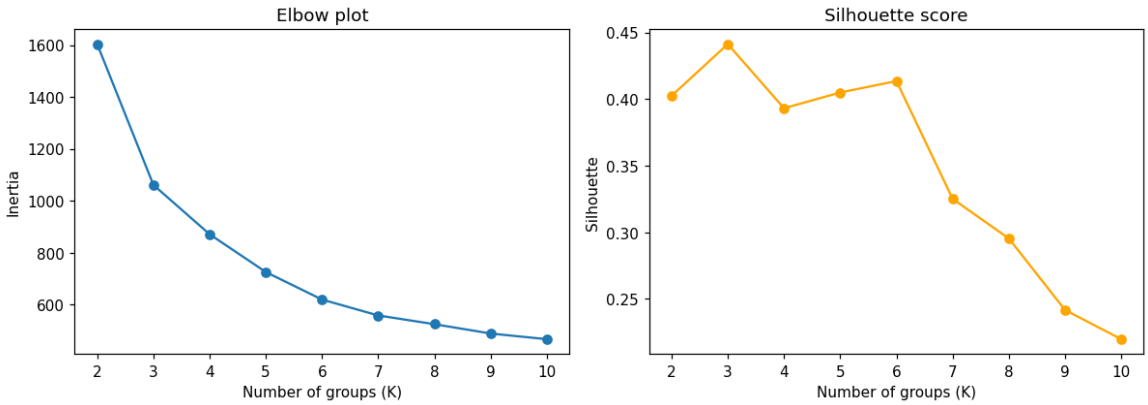
- **Elbow plot (inertia):** inertia measures how spread out the groups are; lower is tighter. It *always* falls as K grows, so we look for the **elbow** – the point where the curve stops falling steeply.
- **Silhouette score:** from −1 to +1. Higher means customers sit comfortably inside their own group and far from other groups. Look for the **highest peak**.

```
# ===== PART D: K-means =====
# Step 1: try K = 2 to 10 and record two measures
inertia, silhouette = [], []
for k in range(2, 11):
    model = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled)
    inertia.append(model.inertia_)                          # how spread out the groups are (lower = tighter)
    silhouette.append(silhouette_score(X_scaled, model.labels_))  # how well separated (higher = better)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(range(2, 11), inertia, marker="o")
axes[0].set_title("Elbow plot")
axes[0].set_xlabel("Number of groups (K)")
axes[0].set_ylabel("Inertia")
axes[1].plot(range(2, 11), silhouette, marker="o", color="orange")
axes[1].set_title("Silhouette score")
axes[1].set_xlabel("Number of groups (K)")
axes[1].set_ylabel("Silhouette")
plt.tight_layout()
plt.show()

# Step 2: run K-means with the K you choose (we keep K = 4 from Part B)
kmeans = KMeans(n_clusters=K, n_init=10, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Step 3: describe each group by its average values
df["KMeans_Group"] = kmeans_labels
print(df["KMeans_Group"].value_counts().sort_index())
print(df.groupby("KMeans_Group")[features].mean().round(1))
```



Sample output: elbow plot (left) and silhouette score (right) for K = 2 to 10.

What you should see: two curves, then group sizes and a table of the average Age, Income, Spending etc. in each group. This table is the **cluster profile**: read across each row to describe the group, for example “young, low income, buys mostly online”.

STUDENT TASK

- Do the elbow and the silhouette suggest the same K? If not, which do you trust, and why?
- Set K to your final choice, run the block again and give each group a **name** based on its average values (write your own – do not copy anyone else's).

Part E. Method 4: DBSCAN

Idea in plain words: DBSCAN looks for **crowded areas**. It has two settings:

- **eps:** the size of the circle drawn around each customer ("how close counts as a neighbour?").
- **min_samples:** how many customers must be inside the circle for the area to count as crowded.

Type of customer	Meaning
Core point	Has at least *min_samples* customers in its circle – the heart of a crowd.
Border point	Not crowded itself, but lies in the circle of a core point – the edge of a crowd.
Noise point	Neither of the above – standing alone. DBSCAN labels these **−1** and does not force them into a group.

Two nice things: you **do not choose the number of groups** (it is a result), and DBSCAN **finds unusual customers** (noise). The catch: the result depends strongly on eps and min_samples, so we try several values.

```
# ===== PART E: DBSCAN =====
# Step 1: try different "neighbourhood sizes" (eps) and see what happens
print("eps  min_samples  groups  noise_customers")
for eps in [0.6, 0.8, 1.0, 1.2]:
    for min_samples in [5, 10]:
        labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X_scaled)
        n_groups = len(set(labels)) - (1 if -1 in labels else 0)   # -1 is not a group, it means "noise"
        n_noise = (labels == -1).sum()
        print(f"{eps:<4} {min_samples:<12} {n_groups:<7} {n_noise}")

# Step 2: choose YOUR final values and fit the model
EPS = 0.9            # <-- YOUR DECISION
MIN_SAMPLES = 8      # <-- YOUR DECISION
db_labels = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES).fit_predict(X_scaled)

print("\nGroups found:", len(set(db_labels)) - (1 if -1 in db_labels else 0))
print("Noise customers:", (db_labels == -1).sum())
```

```
eps  min_samples  groups  noise_customers
0.6  5            4       260
0.6  10           2       338
0.8  5            3       109
0.8  10           4       197
1.0  5            1       27
1.0  10           1       52
1.2  5            1       6
1.2  10           1       7

Groups found: 3
Noise customers: 88
```

What do these results teach?

- **eps too small** (0.6): the circles are tiny, almost nobody is "crowded", so hundreds of customers become noise.
- **eps too large** (1.2): the circles are so big that all crowds join together – just one group and almost no noise.
- **Bigger min_samples**: a stricter definition of crowded, so more noise.

STUDENT TASK

- Choose your own EPS and `MIN_SAMPLES` (try values between the ones in the table) so that you get a sensible number of groups and not too much noise. Explain your choice.
- Print the noise customers with `df[db_labels == -1].head(10)`. What kind of customers are they?

Part F. Compare All Four Methods

Drawing the groups. We cannot draw six dimensions, so we use **PCA** to squeeze the six variables into two for the picture only. The clustering itself was done on all six variables; the picture is just a convenient view.

Scoring the groups. We use the silhouette score again. DBSCAN's noise customers are left out when scoring, because they belong to no group.

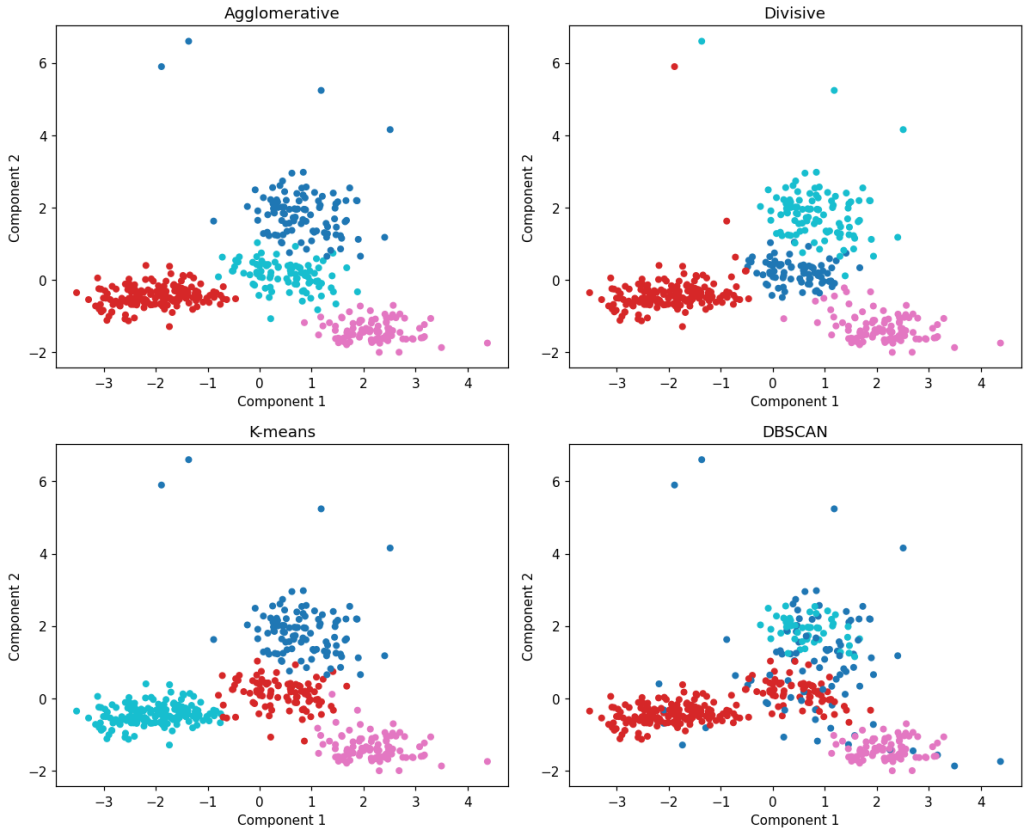
```
# ===== PART F: Compare all four methods =====
# Step 1: squeeze the 6 variables into 2 for drawing (PCA is ONLY for the picture)
X_2d = PCA(n_components=2, random_state=42).fit_transform(X_scaled)

methods = {"Agglomerative": agg_labels, "Divisive": div_labels,
           "K-means": kmeans_labels, "DBSCAN": db_labels}

fig, axes = plt.subplots(2, 2, figsize=(11, 9))
for ax, (name, labels) in zip(axes.ravel(), methods.items()):
    ax.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap="tab10", s=18)
    ax.set_title(name)
    ax.set_xlabel("Component 1")
    ax.set_ylabel("Component 2")
plt.tight_layout()
plt.show()      # for DBSCAN, noise customers (-1) get their own colour

# Step 2: a score table (silhouette: higher = better separated groups)
rows = []
for name, labels in methods.items():
    keep = labels != -1                              # ignore DBSCAN noise when scoring
    n_groups = len(np.unique(labels[keep]))
    score = silhouette_score(X_scaled[keep], labels[keep]) if n_groups > 1 else np.nan
    rows.append([name, n_groups, int((~keep).sum()), round(score, 3)])

print(pd.DataFrame(rows, columns=["Method", "Groups", "Noise customers", "Silhouette"]))
```



Sample output: the four clusterings drawn on the same two-dimensional map (DBSCAN noise has its own colour).

```
          Method  Groups  Noise customers  Silhouette
0  Agglomerative       4                0       0.388
1       Divisive       4                0       0.378
2        K-means       4                0       0.393
3         DBSCAN       3               88       0.442
```

(Sample for the starter choices K = 4, eps = 0.9, min_samples = 8. Your numbers will depend on your choices.)

Careful: the highest score is not automatically the best method!

- DBSCAN's score looks high partly because it **ignored 88 customers**. The other methods had to place everyone.
- A good method also gives groups you can **understand and use**. Can you name each group? Is each group big enough to matter?

STUDENT TASK

- Fill in the table below using your own results.
- Write a paragraph (5–8 lines): **which method do you recommend for this dataset, and why?** Mention both the scores and how easy the groups are to describe.

Method	Groups	Noise	Silhouette	One advantage	One limitation
Agglomerative					
Divisive					
K-means					
DBSCAN					

Quick Summary of the Four Methods

Method	Must I choose the number of groups?	Finds noise?	Main settings
Agglomerative	No – choose by cutting the dendrogram	No	Linkage (e.g. Ward), where to cut
Divisive	Stop when you have enough groups	No	Splitting rule, number of groups
K-Means	**Yes** (K)	No	K
DBSCAN	**No**	**Yes**	eps, min_samples

What to Submit

Submit your notebook (code, outputs and graphs) together with short written answers for these items:

1. Dataset facts (customers, columns, missing values) – Part A, Step 1.
2. How you cleaned the data and why.
3. Why we standardize.
4. Agglomerative: your dendrogram, the number of groups you chose, and why.
5. Divisive: group sizes and a comparison with agglomerative.
6. K-means: elbow and silhouette plots, your chosen K, and a **name for each group**.
7. DBSCAN: your chosen eps and min_samples, and what the noise customers look like.
8. The comparison table and your final recommendation (Part F).

Short quiz (answer in one or two lines each)

1. What is clustering, and why is it called unsupervised learning?
2. Why do we exclude Customer_ID?
3. What is a dendrogram?
4. What is the difference between agglomerative and divisive clustering?
5. What does inertia measure? Why does it keep falling as K grows?
6. What does a silhouette score close to 1 mean?
7. What is a noise point in DBSCAN?
8. What happens if eps is too small? Too large?
9. Why can the method with the highest silhouette score still be the wrong choice?

Glossary

Word	Plain meaning
Cluster	A group of similar customers.
Distance	How far apart two customers are (more different = larger distance).
Standardization	Rescaling each variable to average 0 and spread 1 so that all variables count equally.
Dendrogram	A tree diagram showing the order in which groups were joined.
Linkage	The rule for measuring distance between two groups.
Centroid	The centre (average position) of a group.
Inertia	How spread out the customers are around their centroids. Lower = tighter groups.
Silhouette score	A score from −1 to 1 for how well separated the groups are. Higher is better.
Noise	Customers DBSCAN does not put in any group (label −1).
PCA	A trick to squeeze many variables into two so we can draw a picture.

If something goes wrong

Message / problem	What to do
FileNotFoundError: customer_segmentation.csv	The CSV is not in the same folder as your notebook. Move it there (in Colab, upload it again).
ModuleNotFoundError: No module named 'sklearn'	Run `pip install scikit-learn` (the name differs from the import name).
NameError: name 'X_scaled' is not defined	You skipped a block. Run the blocks **in order** from the top (Restart & Run All).
No graph appears	In a plain Python file add `plt.show()`; in Jupyter run the cell again.
My numbers differ slightly from the sample	Normal if you changed K, eps or min_samples. With the same starter values you should match closely.